Semantic Non-Axiomatic Reasoning System (SeNARS)

A Hybrid Neuro-Symbolic Platform for Cognitive Architecture Research

SeNARS Developers github.com/automenta/senars11

Preprint • CC-BY-4.0 • Source: AGPL-3.0

Abstract. SeNARS is a hybrid neuro-symbolic reasoning system that integrates Non-Axiomatic Logic (NAL) with large language models (LLMs) to create an observable, stream-based platform for cognitive architecture research. Unlike pure symbolic systems that struggle with real-world ambiguity, or pure neural approaches that lack interpretability and consistency, SeNARS combines the formal rigor of NAL with the pattern recognition capabilities of modern language models.

The architecture implements a continuous, non-blocking dataflow pipeline that processes streams of premises into conclusions while maintaining epistemic stability through explicit truth values and evidence tracking. Symbolic NAL rules execute synchronously with rigorous truth-value propagation; LLM queries dispatch asynchronously through circuit-breaker-protected channels. Dual memory separates short-term attention from long-term storage with automated consolidation.

Crucially, this paper describes a *deliberately incomplete* system—designed as substrate for an ecosystem of cognitive architectures rather than a finished application. We argue that minimal, extensible foundations enable more diverse and innovative applications than complete but rigid systems.

Keywords: neuro-symbolic AI, cognitive architecture, non-axiomatic logic, hybrid reasoning, stream processing, deliberate incompleteness, epistemic uncertainty

1. Introduction and Motivation

The field of artificial intelligence faces a fundamental tension between two dominant paradigms: symbolic AI systems that provide formal guarantees and interpretability but struggle with real-world complexity, and neural network approaches that excel at pattern recognition but lack transparency and consistency. This paper introduces SeNARS (Semantic Non-Axiomatic Reasoning System), a hybrid architecture designed to bridge this divide.

1.1. Limitations of Pure Symbolic NARS

The Non-Axiomatic Reasoning System (NARS), developed by Pei Wang, represents a significant departure from classical logic systems. Rather than assuming complete and consistent knowledge, NARS operates under the Assumption of Insufficient Knowledge and Resources (AIKR)—recognizing that any real-world system must reason with incomplete, uncertain, and potentially contradictory information under finite computational resources.

Despite its theoretical elegance, pure symbolic NARS implementations face practical limitations:

- **Knowledge Acquisition**: Converting natural language knowledge into formal Narsese representations requires significant manual effort.
- **Grounding Problem**: Atomic terms lack inherent semantic content, relying entirely on structural relationships.
- **Scaling Challenges**: Building comprehensive knowledge bases with controlled vocabularies is labor-intensive.
- **Natural Language Interface**: Users must learn Narsese syntax to interact effectively.

1.2. Limitations of Pure LLM Approaches

Large language models have demonstrated remarkable capabilities in natural language understanding, generation, and even reasoning tasks. However, they exhibit fundamental limitations for autonomous agent applications:

- **Epistemic Instability**: LLMs produce different outputs for semantically equivalent queries, making them unreliable for maintaining consistent beliefs.

- **Lack of Persistence**: Without external memory, LLMs have no mechanism for accumulating and revising knowledge over time.
- **No Intrinsic Goals**: LLMs are purely reactive, completing patterns without autonomous drives or objectives.
- **Hallucination**: LLMs confidently generate plausible but false statements, with no mechanism for uncertainty quantification.
- **Opacity**: The reasoning process within LLMs remains largely uninterpretable.

1.3. The SeNARS Solution

SeNARS addresses these complementary weaknesses through a hybrid architecture where:

1. **NAL provides the skeleton**—persistent memory, formal truth values, goal-directed behavior, and epistemic consistency.
2. **LLMs provide the flesh**—natural language processing, semantic grounding, world knowledge, and pattern recognition.

1.4. Vision: An Ecosystem of Minds

Imagine a future where cognitive architectures are as diverse as biological species. A medical diagnostic system might emphasize caution and explanation; an educational tutor might favor Socratic questioning; a creative assistant might embrace exploratory leaps. All would share common primitives—truth values, memory consolidation, hybrid inference—while diverging in strategy, domain knowledge, and personality.

SeNARS aspires to be the common ancestor: a minimal, coherent substrate from which this ecosystem can evolve. Not a product to be consumed, but a seed to be grown.

2. Background

2.1. Non-Axiomatic Logic and NARS

Non-Axiomatic Logic (NAL) is a formal logic system designed for reasoning under uncertainty with finite resources. Unlike classical logics that assume complete knowledge and unlimited computational resources, NAL embraces the Assumption of Insufficient Knowledge and Resources (AIKR).

NAL represents knowledge through **terms** connected by **copulas**, with associated **truth values**:

- **Inheritance (→)**: "A → B" means A is a specialization of B
- **Similarity (↔)**: Bidirectional similarity relationship
- **Implication (⇒)**: Conditional relationship between statements

Truth values consist of two components:

- **Frequency (f)**: The proportion of positive evidence (range 0–1)
- **Confidence (c)**: The reliability of the frequency estimate (range 0–1)

This representation, written as {f, c}, enables principled reasoning through truth value revision rules that combine evidence while accounting for overlapping sources through **evidential stamps**.

NARS distinguishes task types by punctuation: **Beliefs (.)** for declarative knowledge, **Goals (!)** for desired states, and **Questions (?)** for queries.

OpenNARS demonstrated these principles in research settings. ANSNA explored a minimal implementation for embedded systems. SeNARS continues this lineage, reimagining the architecture for modern streaming systems and neural integration.

2.2. The AIKR Principle

The **Assumption of Insufficient Knowledge and Resources** (AIKR) is the foundational premise of NAL. Unlike classical logics that assume complete, consistent knowledge, AIKR acknowledges that any real intelligent agent operates under fundamental constraints:

Insufficient Knowledge. The agent never has complete information. Facts may be unknown, uncertain, or contradictory. No closed-world assumption is valid.

Insufficient Resources. Computation, memory, and time are always finite. Exhaustive search is impossible.

Real-time Demands. Decisions must be made before complete analysis is possible. Waiting for certainty means missing opportunities.

AIKR has profound implications for system design:

- **Graded truth**: Beliefs carry explicit uncertainty rather than binary true/false
- **Evidence accumulation**: New observations revise rather than replace existing beliefs
- **Anytime reasoning**: Inference produces useful partial results at any interruption
- **Resource-bounded**: Computation allocated by importance, not exhaustive search

- **Graceful degradation**: Quality degrades smoothly under resource pressure

Every architectural decision in SeNARS reflects AIKR: truth values quantify uncertainty, priority-based sampling allocates attention, bounded Focus forces consolidation, streaming produces conclusions continuously, and circuit breakers ensure responsiveness.

2.3. LLM Reasoning Advances

Recent work has shown that Large Language Models can approximate structured reasoning when properly prompted. Chain-of-Thought elicits step-by-step reasoning; ReAct interleaves reasoning with actions; Reflexion introduces verbal self-critique; Toolformer teaches models to invoke external APIs.

These techniques demonstrate that LLMs can *simulate* structured reasoning. But they lack formal grounding: no explicit uncertainty quantification, no persistent memory across sessions, no guarantee of logical consistency. SeNARS provides the missing infrastructure, using NAL as the backbone and LLMs as a knowledge-rich oracle.

3. Architecture

SeNARS implements a **streaming dataflow architecture** that continuously transforms observations and queries into conclusions and actions. The design emphasizes immutability for correctness, non-blocking execution for responsiveness, and explicit extension points for adaptability.

3.1. Design Principles

1. **Immutable Data Foundation**: Core data structures (Terms, Tasks, Truth, Stamps) are immutable, ensuring consistency and enabling efficient caching.
2. **Component-Based Architecture**: All major components inherit from a common base with standardized lifecycle (initialize, start, stop, dispose).
3. **Pipeline-Based Processing**: Continuous stream architecture for processing premises into conclusions.
4. **Resource Awareness**: CPU throttling, backpressure handling, and derivation depth limits.
5. **Observable Reasoning**: Complete visibility into reasoning processes through event-driven communication.

3.2. Core Data Structures

Terms. The fundamental unit of knowledge representation. A Term may be atomic (a symbol like `bird`) or compound (a relation like `bird → animal`). The `TermFactory` normalizes equivalent terms and caches them for reuse.

Truth Values. Every statement carries a truth value comprising frequency (how often true) and confidence (evidential strength). Truth values propagate through inference according to NAL semantics.

Stamps. Evidence tracking for cyclicity detection, revision, and provenance. Each Stamp records a unique identifier, occurrence time, and source.

Tasks. The unit of processing. A Task contains a Term, truth value, stamp, priority, and punctuation indicating its type.

3.3. The Streaming Pipeline

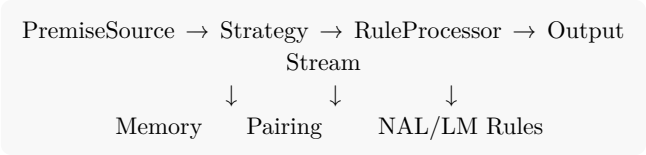


Figure 1: The SeNARS streaming dataflow pipeline.

PremiseSource generates tasks from memory using configurable sampling objectives:

- Priority-based sampling (higher priority tasks selected more often)
- Recency-based sampling (favor recently updated information)
- Punctuation-focused sampling (prioritize goals or questions)
- Novelty sampling (favor tasks with fewer derivation steps)

Strategy components create premise pairs for reasoning:

- **BagStrategy**: NARS-style priority-sampled approach for anytime reasoning
- **ExhaustiveStrategy**: Comprehensive search for all related beliefs
- **PrologStrategy**: Goal-driven backward chaining with unification
- **ResolutionStrategy**: Prolog-like resolution for question answering

RuleProcessor executes inference rules in a hybrid manner:

- Synchronous NAL rules execute immediately
- Asynchronous LM rules dispatch without blocking

- Results merge into a unified output stream

3.4. LM Integration Rules

Rule	Function
Elaboration	Generate related beliefs from a concept
WorldKnowledge	Query factual knowledge to fill gaps
Translation	Convert natural language to Narsese
Explanation	Generate natural language from derivations
Disambiguation	Resolve ambiguous terms using context

Table 1: Language Model Integration Rules

3.5. Dual Memory Architecture

SeNARS implements a biologically-inspired dual memory system:

Focus (Short-term). High-priority, limited-capacity memory for immediate processing. The Focus system implements attention: selecting which tasks receive reasoning resources based on priority, recency, and diversity constraints.

Long-term Memory. Persistent storage for all Concepts (collections of Tasks sharing a common Term) with specialized indexes for efficient retrieval.

Consolidation. Intelligent movement between memory types based on priority and usage. Configurable forgetting policies manage memory pressure.

3.6. Layer System

The Layer system manages semantic connections between concepts, providing infrastructure for both explicit and implicit relationships:

TermLayer. Explicit connections between terms with priority-based management. Connections are typed (inheritance, similarity, implication) and carry weights that decay over time without reinforcement.

EmbeddingLayer. Vector embeddings for semantic similarity computation. Enables approximate matching when exact term matches fail, bridging the gap between symbolic precision and neural flexibility.

3.7. Hybrid NAL-LLM Integration

The key architectural innovation is **non-blocking hybrid execution**:

- **NAL rules** execute synchronously with rigorous truth-value propagation.
- **LM rules** execute asynchronously; results integrate with assigned confidence values reflecting source reliability.
- **Circuit breakers** detect LM failures (timeouts, rate limits, errors) and automatically fall back to pure symbolic reasoning.
- **Validation gates** allow NAL constraints to filter or adjust LM outputs before integration, maintaining epistemic consistency.

Epistemic Anchoring: When the LM generates information that contradicts established high-confidence beliefs, the NAL system can reject or down-weight the LM output. For example, if SeNARS holds the belief "(fire → hot) {1.0, 0.9}", an LM hallucination suggesting fire is cold would be rejected.

3.8. The Cognitive Operating System Analogy

We conceptualize the NAL-LLM relationship through an "operating system" analogy: the LLM functions as a powerful **CPU/ALU** that can rapidly process symbols and patterns but has no persistent state. The NAL system acts as the **kernel**:

- **Scheduler**: The reasoning pipeline determines which "processes" receive LLM compute time.
- **Filesystem**: Memory and Term structures provide persistent storage of beliefs and goals.
- **Permissions**: Truth values and evidence stamps determine what information is trusted.
- **Goals**: The belief/goal distinction provides intentionality that reactive LLMs lack.

Where LLMs are fluid and context-dependent, SeNARS provides the *anchor*—stable beliefs that persist across sessions and constrain ephemeral neural outputs.

4. Prototype Implementation

The current SeNARS prototype is implemented in JavaScript (Node.js), emphasizing accessibility and rapid iteration.

4.1. Technology Stack

Core Engine Stream-based reasoning pipeline with configurable components

Memory System Dual-layer architecture with automated consolidation

LM Integration Provider registry supporting OpenAI, Anthropic, Ollama, HuggingFace

Parser Complete Narsese parser for NAL syntax

Web UI React-based visualization for reasoning traces

Monitoring WebSocket-based event streaming

4.2. Component Status

Component	Status
Core reasoning engine (NAR)	Complete
Stream reasoner pipeline	Complete
Dual memory architecture	Complete
Truth value operations	Complete
Narsese parser	Complete
Multiple strategies (Bag, Prolog, Resolution)	Complete
LM provider integration	Complete
Circuit breaker pattern	Complete
Event-driven architecture	Complete
Layer system (Term, Embedding)	Complete

Table 2: Core component implementation status.

4.3. Codebase Architecture

Directory	Purpose
Core Library (core/src/)	
`reason/`	Reasoner, RuleProcessor, Strategy, PremiseSource
`rules/`	NAL inference rules (deduction, induction, etc.)
`strategy/`	BagStrategy, PrologStrategy, ExhaustiveStrategy
`memory/`	Memory, Focus, Bag, Concept, MemoryIndex
`indexes/`	Specialized term/task indexes
`lm/`	LM providers, NarseseTranslator, EmbeddingLayer
`term/`	Term, TermFactory, normalization
`task/`	Task, priority, punctuation
`parser/`	Narsese tokenizer, parser, AST
`util/`	Logging, EventBus, CircuitBreaker, metrics
External Interfaces	
`ui/`	React web UI for visualization
`repl/`	Interactive command-line interface
`tests/`	Unit, integration, e2e tests
`examples/`	Usage examples and demos

Table 3: Source Code Directory Structure

4.4. User Interfaces

SeNARS provides multiple interfaces for different use cases:

1. **TUI (Text User Interface)**: Command-line REPL for direct system interaction
2. **Web UI**: React/Vite-based visualization with graph visualization of concepts, reasoning trace panels, task flow diagrams, priority histograms, and real-time concept activation monitoring
3. **WebSocket Monitor**: Real-time system monitoring through web connections

4.5. Performance Metrics

Metric	Relevance
Reasoning Efficiency	
Inferences/sec	Raw symbolic reasoning throughput
Rule match rate	Fraction yielding valid conclusions
Derivation depth	Average chain length
Memory Management	
Concept count	Knowledge breadth in long-term memory
Focus utilization	Attention efficiency
Cache hit ratio	Term normalization effectiveness
Hybrid Integration	
LM query latency	Neural responsiveness
LM success rate	Query reliability
Circuit breaker trips	External service stability
System Responsiveness	
End-to-end latency	Time from input to first output
Memory footprint	Deployment constraints

Table 4: Key metrics for evaluating SeNARS deployments.

4.6. Validation

The prototype includes comprehensive testing:

- Unit tests for all core data structures (Term, Task, Memory, RuleEngine)
- Integration tests for component interaction and system behavior
- Property-based tests for verification of system invariants
- End-to-end reasoning chain validation

4.7. Operational Capabilities

What works today:

- Complete NAL inference with truth propagation across all copula types
- Multiple reasoning strategies with configurable selection
- Priority-based sampling with recency, novelty, and punctuation objectives
- Resource management: CPU throttling, backpressure, derivation depth limits
- LM integration with automatic failover and circuit breaker protection
- Real-time visualization of reasoning traces and concept activation
- Bidirectional natural language ↔ Narsese translation
- WebSocket-based monitoring for remote observation

4.8. Example: Transitive Inference

```
Input:  (bird --> animal){0.90, 0.90}.
Input:  (robin --> bird){0.95, 0.90}.

Derived: (robin --> animal){0.855, 0.81}.
  Rule: Deduction
  From: robin-->bird, bird-->animal
```

The derived truth value reflects both the frequency and confidence of the premises, reduced through inference to reflect the evidential chain.

4.9. Example: Knowledge Discovery

```
Input:  (salmon --> fish){0.95, 0.90}.
Input:  (fish --> aquatic){0.90, 0.85}.
Query:  (salmon --> ?what)?

Derived: (salmon --> aquatic){0.855, 0.77}.
LM Elaboration: (salmon --> edible){0.85, 0.60}.
LM Elaboration: (salmon --> migratory){0.90,
0.55}.
```

The system combines deductive inference (salmon is aquatic) with LM-derived knowledge (edible, migratory). LM contributions carry lower confidence, reflecting their external source—they can be revised as direct evidence accumulates.

5. Deliberate Incompleteness

This section presents what we believe to be a novel contribution to AI system design philosophy: the principle of **deliberate incompleteness**.

5.1. The Anti-Completeness Manifesto

> *"This is not being built to be a finished application. It is being built to be substrate—the common seed for a future industrial ecosystem of cognitive architectures. The less complete it is right now, the more possibilities it can grow into."*

This philosophy represents a deliberate inversion of typical software development practices. Rather than maximizing features before release, SeNARS intentionally leaves aspects "rough, partial, or missing" to enable diverse applications.

5.2. The Calcification Problem

History shows that "finished" systems tend to ossify. Once a platform presents itself as complete, the friction required to modify it increases dramatically:

- Users depend on specific behaviors
- Documentation hardens into dogma
- Communities fragment over backward compatibility
- Innovation migrates to workarounds rather than foundations

Consider the most generative platforms in computing history. Early Unix succeeded precisely because it was *unfinished*—minimal primitives that invited completion. The Linux kernel remains perpetually incomplete. The web browser was deliberately underspecified, spawning an ecosystem of frameworks.

In AI specifically, the most influential contributions are often frameworks rather than finished products. OpenAI Gym provided minimal abstractions for RL environments. Transformers provided attention mechanisms without prescribing applications. Each enabled diverse completions while preserving a common core.

5.3. The Stability Contract

Deliberately incomplete doesn't mean arbitrary. SeNARS maintains stability in three core areas:

1. **Core Reasoning Stream**: The pipeline architecture is stable and well-defined
2. **Observation Contract**: The event system and visibility mechanisms are consistent
3. **Hybrid Integration**: The NAL-LM collaboration pattern is established

These stable foundations provide the "skeleton" upon which diverse "flesh" can grow.

5.4. Invitation to Fork

We envision multiple SeNARS lineages, each completed differently:

Minimal Edge Reasoners Stripped-down implementations for embedded/IoT

High-Agency Planners Extensions for autonomous goal pursuit

Educational Sandboxes Interactive environments for teaching AI reasoning

Personal Memory Layers Lifelong persistent context for individual users

Multi-Agent Societies Distributed reasoning across networks of agents

Alternative Logics Entirely new formal systems beyond NAL

All would share the core pipeline, enabling cross-pollination: a performance optimization from one fork could benefit others; a novel strategy could migrate across applications.

5.5. Extension Points

Strategy New premise-pairing algorithms via Strategy interface

LM Providers Custom integrations via provider registry

Memory Policies Configurable forgetting, consolidation, indexing

Rule Sets Dynamic registration of custom NAL and LM rules

Truth Functions Alternative uncertainty representations

Layers Custom connection types (TermLayer, EmbeddingLayer)

If something you need is not here yet, that is by design. Fork it and grow it into the species you need.

6. Future Directions

6.1. Technical Challenges

Performance Optimization:

- Achieving sub-1ms targets across complete reasoning cycles
- Optimizing memory consolidation for larger knowledge bases
- Reducing latency for LLM integration without sacrificing robustness

Scaling:

- Distributed reasoning across multiple nodes
- Horizontal scaling for large knowledge bases
- Efficient serialization for knowledge transfer between agents

Integration Depth:

- Deeper semantic grounding through embedding layers
- Bidirectional learning between NAL and LM components
- Automatic translation quality improvements between Narsese and natural language

6.2. RLFP: Reinforcement Learning from Preferences

SeNARS incorporates a Reinforcement Learning from Preferences (RLFP) framework: trajectory logging records complete reasoning episodes; preference collection gathers qualitative comparisons between reasoning paths; policy learning guides decision-making in task selection and rule application.

The belief/goal distinction naturally supports this approach. Rather than hand-crafted reward functions, the system learns from qualitative comparisons: "reasoning path A was clearer than path B." This enables learning *how* to think more effectively, not just *what* to think.

6.3. Embodied Grounding

Formal reasoning about "fire → hot" differs from *experiencing* heat. Future work might integrate with robotics platforms or simulation environments to ground symbols in sensorimotor experience, enabling perceptual learning and situated reasoning.

6.4. Lifelong Learning

Open questions: How should retention balance against forgetting as knowledge accumulates? How can new learning integrate without catastrophic interference? How should the system detect and adapt to distributional shift?

6.5. Formal Verification

For safety-critical applications: Can we prove that inferences preserve consistency? Can we bound computation for specific query types? Formal methods could provide guarantees that neither symbolic nor neural systems offer alone.

6.6. Alternative Implementations

The architecture is language-agnostic: **Rust** for performance, **WebAssembly** for browsers, **GPU kernels** for accelerated reasoning.

6.7. Call for Collaborators

We explicitly invite collaboration from:

- **Researchers** interested in formal foundations of hybrid AI
- **Engineers** building practical reasoning systems
- **Educators** developing AI curricula with transparent systems
- **Domain experts** applying reasoning to specific fields
- **Philosophers** examining questions of machine cognition

Clone the repository and explore `examples/`. The Web UI provides immediate visibility into reasoning traces.

6.8. Research Opportunities

Theoretical:

- Formal semantics for NAL-LM hybrid truth values
- Convergence properties under mixed symbolic-neural inference
- Information-theoretic bounds on hybrid reasoning

Empirical:

- Benchmarking against pure symbolic and pure neural baselines
- Human studies on reasoning transparency and trust
- Long-term stability studies for persistent agents

Application Domains:

- Personal knowledge management assistants
- Educational tutoring systems with explainable reasoning
- Scientific hypothesis generation and testing
- Autonomous agents with documented decision rationale
- Medical diagnostic systems with uncertainty quantification

7. Conclusion

SeNARS represents a practical approach to hybrid neuro-symbolic AI that addresses the complementary weaknesses of pure symbolic and pure neural systems. By combining the formal rigor and consistency of Non-Axiomatic Logic with the pattern recognition and natural language capabilities of large language models, we provide a foundation for cognitive architectures that are both powerful and transparent.

The system's stream-based architecture, dual memory system, multiple reasoning strategies, and robust LM integration offer a flexible platform for exploring advanced AI reasoning. Importantly, our principle of deliberate incompleteness positions SeNARS not as a finished product but as a *substrate*—a seed for an ecosystem of cognitive architectures tailored to diverse applications.

We release SeNARS as a public good. We welcome contributions, forks, experiments, and critiques. The goal is not to build the perfect system, but to cultivate the conditions for many perfect systems to emerge.

Fork it, strip it, break it, grow it.
Build the species you need.

Acknowledgments. We acknowledge the foundational work of Pei Wang on Non-Axiomatic Logic and NARS, which provides the theoretical basis for SeNARS. We also thank the broader AGI and neuro-symbolic AI research communities for ongoing discussions that have shaped this work.

8. References

Brown, T. B. et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901.

Gabriel, R. P. (1991). The rise of "worse is better." In *Lisp: Good News, Bad News, How to Win Big*.

Hammer, P. and Lofthouse, T. (2020). ANSNA: An attention-driven non-axiomatic semantic navigation architecture. *AGI*.

Marcus, G. and Davis, E. (2020). Rebooting AI: Building artificial intelligence we can trust. Vintage.

Schick, T. et al. (2023). Toolformer: Language models can teach themselves to use tools. *NeurIPS*.

Shinn, N. et al. (2023). Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*.

Wang, P. (1995). Non-axiomatic reasoning system: Exploring the essence of intelligence. PhD thesis, Indiana University.

Wang, P. (2006). *Rigid Flexibility: The Logic of Intelligence*. Springer.

Wang, P. (2013). *Non-Axiomatic Logic: A Model of Intelligent Reasoning*. World Scientific.

Wei, J. et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*.

Yao, S. et al. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR*.